

Inter-Process Communication in Linux

Semafori, Memoria Condivisa e Meccanismi IPC

Roberto Marino, PhD

Università di Messina
CdL in Informatica — Reti e Sistemi Distribuiti Mod.B

May 11, 2026



UniMe
1548

Il problema

I processi Unix sono **isolati per design**: spazi di indirizzamento separati, nessuna variabile globale condivisa.

Per cooperare devono usare un **canale esplicito** fornito dal kernel.

Tre domande da porsi

- ▶ Devo **trasferire dati** o solo **sincronizzare**?
- ▶ I processi sono **imparentati** (fork) o indipendenti?
- ▶ Serve **persistenza** dopo la terminazione?

Comunicazione (scambio dati)

- ▶ `pipe()` / `mkfifo()` — pipe anonime e FIFO
- ▶ Memoria condivisa — `shm_open` / `shmget`
- ▶ Code di messaggi — POSIX `mq_open`, SysV `msgget`
- ▶ Socket Unix — `AF_UNIX`, bidirezionale

Sincronizzazione

- ▶ Semafori POSIX — con nome e senza nome
- ▶ Semafori System V — `semget` / `semop`
- ▶ Mutex POSIX — con `PROCESS_SHARED`
- ▶ Segnali — `kill` / `sigqueue`

Meccanismo	Dati	Sync	Non-parenti	Persistenza
Pipe anonima	✓	—	—	—
FIFO	✓	—	✓	file
Shared memory	✓	—	✓	opzionale
Code messaggi	✓	—	✓	fino a reboot
Semafori POSIX	—	✓	✓	fino a unlink
Semafori SysV	—	✓	✓	fino a reboot

Caratteristiche

- ▶ Canale **unidirezionale** e **in-kernel**
- ▶ Solo tra processi **imparentati** (dopo **fork**)
- ▶ **Bloccante** per default: **read** attende dati, **write** attende spazio
- ▶ Automaticamente chiusa quando tutti i fd vengono chiusi

Regola fondamentale

Ogni processo deve chiudere il capo della pipe che **non usa**. Altrimenti **read** non riceverà mai EOF.

```
1 int fd[2];
2 pipe(fd);      /* fd[0]=read, fd[1]=write */
3
4 if (fork() == 0) {
5     /* figlio: scrive */
6     close(fd[0]);      /* chiude lettura
7                          */
8     write(fd[1], "ciao", 5);
9     close(fd[1]);
10    exit(0);
11 }
12 /* padre: legge */
13 close(fd[1]);      /* chiude
14                      scrittura */
15 char buf[16];
16 read(fd[0], buf, sizeof(buf));
17 close(fd[0]);
```

FIFO (pipe con nome)

Differenze rispetto alla pipe anonima

- ▶ Esiste come **file speciale** nel filesystem
- ▶ Accessibile da **processi non imparentati**
- ▶ Stessa semantica bloccante della pipe
- ▶ Rimane nel filesystem finché non viene rimossa con `unlink`

Caso d'uso tipico

Due processi avviati indipendentemente che devono scambiarsi un flusso di dati senza voler usare socket.

```
1  /* Processo A: crea e scrive */
2  mkfifo("/tmp/myfifo", 0666);
3  int fd = open("/tmp/myfifo",
4               O_WRONLY);
5  write(fd, "messaggio", 10);
6  close(fd);
7
8  /* Processo B: legge */
9  int fd = open("/tmp/myfifo",
10               O_RDONLY);
11 char buf[64];
12 read(fd, buf, sizeof(buf));
13 close(fd);
14
15 /* Pulizia */
16 unlink("/tmp/myfifo");
```

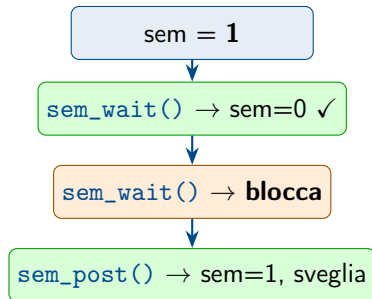
Definizione

Un semaforo è un **contatore intero non negativo** gestito dal kernel su cui sono definite due operazioni atomiche:

- ▶ `wait (P, sem_wait)`: **decrementa**; se il valore è 0, **blocca** il chiamante
- ▶ `post (V, sem_post)`: **incrementa**; se ci sono processi/thread in attesa, ne **sveglia uno**

Attenzione

A differenza del mutex, chiunque può fare `sem_post`: non esiste il concetto di **ownership**.



Caratteristiche

- ▶ Dichiarati come variabile di tipo `sem_t`
- ▶ `sem_init(&s, 0, val)` — il secondo parametro `pshared=0` li limita al **processo corrente**
- ▶ Vivono in memoria normale: stack, heap, globale
- ▶ Distrutti con `sem_destroy(&s)`

Uso tipico

Sincronizzare due thread: uno produce, l'altro consuma. Il semaforo conta gli **elementi disponibili**.

```
1 #include <semaphore.h>
2 #include <pthread.h>
3
4 sem_t sem;
5 int dato = 0;
6
7 void *produttore(void *) {
8     dato = 42;
9     sem_post(&sem);    /* segnala */
10    return NULL;
11 }
12 void *consumatore(void *) {
13     sem_wait(&sem);    /* attende */
14     printf("%d\n", dato);
15     return NULL;
16 }
17 int main(void) {
18     sem_init(&sem, 0, 0); /* val=0 */
19     pthread_t p, c;
20     pthread_create(&p, NULL, produttore, NULL);
21     pthread_create(&c, NULL, consumatore, NULL);
22     pthread_join(p, NULL);
23     pthread_join(c, NULL);
24     sem_destroy(&sem);
25 }
```

Caratteristiche

- ▶ Identificati da un **nome** tipo `"/mio_sem"`
- ▶ Vivono in `/dev/shm`, accessibili da qualsiasi processo
- ▶ `sem_open(name, O_CREAT, mode, val)` crea o apre il semaforo
- ▶ Sopravvivono alla terminazione finché non si fa `sem_unlink`
- ▶ Ogni processo che lo usa deve fare `sem_close`

Pulizia obbligatoria

`sem_close` chiude l'handle locale. `sem_unlink` rimuove il semaforo dal sistema. Senza `unlink` rimane in `/dev/shm` dopo il reboot.

```
1  /* Processo A: crea e segnala */
2  sem_t *s = sem_open("/sync",
3                      O_CREAT, 0666, 0);
4  /* ... fai lavoro ... */
5  sem_post(s);
6  sem_close(s);
7
8  /* Processo B: attende */
9  sem_t *s = sem_open("/sync",
10                     0, 0, 0); /* solo apri */
11 sem_wait(s);
12 /* ... lavoro dopo sync ... */
13 sem_close(s);
14 sem_unlink("/sync"); /* rimuovi */
```

Funzione	Scopo
<code>sem_open</code>	crea/apre
<code>sem_wait</code>	P (decrementa)
<code>sem_post</code>	V (incrementa)
<code>sem_close</code>	chiude handle
<code>sem_unlink</code>	rimuove

Requisiti

1. Il mutex deve essere in **memoria condivisa**
2. L'attributo `PTHREAD_PROCESS_SHARED` deve essere impostato prima di `pthread_mutex_init`

Differenza cruciale con i semafori

Il mutex ha **ownership**: solo il thread/processo che ha fatto `lock` può fare `unlock`. Violare questa regola è undefined behavior.

Limitazione

Se il processo che detiene il lock termina in modo anomalo, il mutex rimane **bloccato per sempre** (deadlock).

```
1 #include <pthread.h>
2 #include <sys/mman.h>
3
4 typedef struct {
5     pthread_mutex_t mtx;
6     int contatore;
7 } Shared;
8
9 Shared *sh = mmap(NULL, sizeof(Shared),
10     PROT_READ|PROT_WRITE,
11     MAP_SHARED|MAP_ANONYMOUS, -1, 0);
12
13 pthread_mutexattr_t attr;
14 pthread_mutexattr_init(&attr);
15 pthread_mutexattr_setpshared(&attr,
16     PTHREAD_PROCESS_SHARED);
17 pthread_mutex_init(&sh->mtx, &attr);
18 pthread_mutexattr_destroy(&attr);
19
20 if (fork() == 0) {
21     pthread_mutex_lock(&sh->mtx);
22     sh->contatore++;
23     pthread_mutex_unlock(&sh->mtx);
24     exit(0);
25 }
26 wait(NULL);
27 printf("cnt=%d\n", sh->contatore);
28 pthread_mutex_destroy(&sh->mtx);
29 munmap(sh, sizeof(Shared));
```

Ownership: mutex vs. semaforo binario

Mutex, ownership enforced

```
1 pthread_mutex_t mtx;
2 pthread_mutexattr_t attr;
3 pthread_mutexattr_init(&attr);
4 pthread_mutexattr_settype(&attr,
5     PTHREAD_MUTEX_ERRORCHECK);
6 pthread_mutex_init(&mtx, &attr);
7 /* --- Thread A --- */
8 pthread_mutex_lock(&mtx);    /* owner = A */
9 /* sezione critica */
10 pthread_mutex_unlock(&mtx); /* OK: e' A */
11 /* --- Thread B (contemporaneo) --- */
12 pthread_mutex_lock(&mtx);    /* blocca: A e'
13     owner */
13 pthread_mutex_unlock(&mtx); /* OK: ora e' B */
14 /* --- Thread B fa unlock senza lock --- */
15 int r = pthread_mutex_unlock(&mtx);
16 /* r == EPERM: B non e' il proprietario */
```

Semaforo binario, no ownership

```
1 sem_t sem;
2 sem_init(&sem, 0, 1); /* val = 1 */
3 /* --- Thread A --- */
4 sem_wait(&sem); /* contatore: 1->0 */
5 /* sezione critica */
6 sem_post(&sem); /* contatore: 0->1 */
7 /* --- Thread B fa post SENZA wait --- */
8 sem_post(&sem); /* contatore: 1->2 !! */
9 /* nessun errore: il kernel non sa
10     chi ha fatto sem_wait */
11 /* Risultato: due thread possono ora
12     entrare contemporaneamente nella
13     sezione critica -> race condition */
14 sem_wait(&sem); /* Thread C: passa! */
15 sem_wait(&sem); /* Thread D: passa! */
```

Il mutex tiene traccia del **proprietario** e rifiuta unlock non autorizzati. Il semaforo tiene solo un **contatore**: chiunque può incrementarlo,

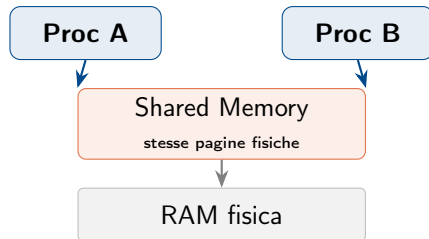
Memoria condivisa: il meccanismo più veloce

Perché è il più veloce?

Con pipe e socket il kernel **copia i dati** da un buffer all'altro. Con la memoria condivisa i due processi vedono **le stesse pagine fisiche**: zero copie, zero chiamate di sistema per ogni accesso.

Attenzione: no sincronizzazione implicita

La memoria condivisa trasferisce dati ma **non coordina** gli accessi. Senza semafori o mutex le scritture concorrenti causano **race condition**.



Pattern di utilizzo

1. `shm_open(name, O_CREAT|O_RDWR, 0666)`
— crea oggetto
2. `ftruncate(fd, size)` — imposta dimensione
3. `mmap(..., fd, 0)` — mappa in spazio virtuale
4. Uso della memoria (con sincronizzazione esterna)
5. `munmap, close, shm_unlink`

Dove vive?

In `/dev/shm`: un filesystem in RAM montato dal kernel. Visibile con `ls /dev/shm`.

```
1  /* Processo A: crea e scrive */
2  int fd = shm_open("/myshm",
3      O_CREAT|O_RDWR, 0666);
4  ftruncate(fd, sizeof(int));
5  int *p = mmap(NULL, sizeof(int),
6      PROT_READ|PROT_WRITE,
7      MAP_SHARED, fd, 0);
8  *p = 99;
9  munmap(p, sizeof(int));
10 close(fd);
11
12 /* Processo B: legge */
13 int fd = shm_open("/myshm",
14     O_RDONLY, 0);
15 int *p = mmap(NULL, sizeof(int),
16     PROT_READ,
17     MAP_SHARED, fd, 0);
18 printf("letto: %d\n", *p);
19 munmap(p, sizeof(int));
20 close(fd);
21 shm_unlink("/myshm");
```

Memoria condivisa con mmap anonima

Quando usarla

Solo tra processi **imparentati** (padre-figlio dopo **fork**). Non serve un nome: la mappatura è ereditata automaticamente dal figlio con le stesse pagine fisiche.

Flag essenziali

- ▶ **MAP_SHARED**: le scritture sono visibili agli altri
- ▶ **MAP_ANONYMOUS**: non associata a nessun file (fd=-1)

MAP_PRIVATE vs MAP_SHARED

Con **MAP_PRIVATE** ogni processo ha una **copia** (Copy-on-Write): le scritture non sono visibili agli

```
1 #include <sys/mman.h>
2 #include <sys/wait.h>
3
4 int *counter = mmap(NULL,
5     sizeof(int),
6     PROT_READ|PROT_WRITE,
7     MAP_SHARED|MAP_ANONYMOUS,
8     -1, 0);
9 *counter = 0;
10
11 for (int i = 0; i < 4; i++) {
12     if (fork() == 0) {
13         (*counter)++;    /* race! */
14         exit(0);
15     }
16 }
17 for (int i = 0; i < 4; i++)
18     wait(NULL);
19
20 /* Valore finale: 1..4 (non deterministico
21    senza sincronizzazione) */
22 printf("%d\n", *counter);
23 munmap(counter, sizeof(int));
```

Il pattern corretto

Shared memory e semafori si **usano insieme**: la memoria trasporta i dati, il semaforo coordina gli accessi.

Struttura tipica in memoria condivisa

```
typedef struct { sem_t mutex; sem_t full; sem_t empty;  int
buffer[N]; int head, tail; } SharedQueue;
```

Mettere i semafori **dentro** la struttura condivisa evita di dover gestire nomi separati per ogni semaforo. Ricordarsi di usare `sem_init` con `pshared=1`.

```
1  /* alloca struttura condivisa */
2  SharedQueue *q = mmap(NULL,
3      sizeof(SharedQueue),
4      PROT_READ|PROT_WRITE,
5      MAP_SHARED|MAP_ANONYMOUS,
6      -1, 0);
7
8  sem_init(&q->mutex, 1, 1);
9  sem_init(&q->full, 1, 0);
10 sem_init(&q->empty, 1, N);
11
12 /* produttore */
13 sem_wait(&q->empty);
14 sem_wait(&q->mutex);
15 q->buffer[q->tail] = item;
16 q->tail = (q->tail+1) % N;
17 sem_post(&q->mutex);
18 sem_post(&q->full);
19
20 /* consumatore */
21 sem_wait(&q->full);
22 sem_wait(&q->mutex);
23 int v = q->buffer[q->head];
24 q->head = (q->head+1) % N;
25 sem_post(&q->mutex);
26 sem_post(&q->empty);
```

Caratteristiche

- ▶ Identificate da un **nome** tipo `"/myqueue"`
- ▶ Il kernel gestisce la sincronizzazione: no mutex espliciti
- ▶ I messaggi hanno una **priorità** (interi senza segno): quelli con priorità più alta escono prima
- ▶ Notifica asincrona con `mq_notify`
- ▶ Linking con `-lrt`

Vantaggio rispetto alle pipe

I messaggi mantengono i **confini**: una `mq_send` corrisponde esattamente a una `mq_receive`. Con le pipe i dati sono un flusso continuo di byte.

```
1 #include <mqqueue.h>
2
3 /* mittente */
4 mqd_t mq = mq_open("/myq",
5                     O_CREAT|O_WRONLY, 0666, NULL);
6 char msg[] = "hello";
7 mq_send(mq, msg, sizeof(msg),
8         0 /* priorit  */);
9 mq_close(mq);
10
11 /* ricevente */
12 mqd_t mq = mq_open("/myq",
13                     O_RDONLY, 0, NULL);
14 struct mq_attr attr;
15 mq_getattr(mq, &attr);
16 char buf[attr.mq_msgsize];
17 unsigned int prio;
18 mq_receive(mq, buf,
19            attr.mq_msgsize, &prio);
20 printf("msg: %s\n", buf);
21 mq_close(mq);
22 mq_unlink("/myq");
```

Caratteristiche

- ▶ Stessa API dei socket TCP/UDP ma solo **in locale**
- ▶ **Bidirezionale**: sia SOCK_STREAM che SOCK_DGRAM
- ▶ Indirizzo = **path nel filesystem** (es. /tmp/my.sock)
- ▶ Più efficienti dei socket TCP/IP per IPC locale
- ▶ Funzionalità unica: passaggio di **file descriptor** tra processi con SCM_RIGHTS

```
1 #include <sys/un.h>
2
3 /* server */
4 int s = socket(AF_UNIX,
5               SOCK_STREAM, 0);
6 struct sockaddr_un addr = {
7     .sun_family = AF_UNIX,
8     .sun_path   = "/tmp/my.sock"
9 };
10 bind(s, (struct sockaddr*)&addr,
11        sizeof(addr));
12 listen(s, 5);
13 int c = accept(s, NULL, NULL);
14 char buf[64];
15 read(c, buf, sizeof(buf));
16 printf("%s\n", buf);
17 close(c); close(s);
18 unlink("/tmp/my.sock");
19
20 /* client */
21 int s = socket(AF_UNIX,
22               SOCK_STREAM, 0);
23 connect(s, (struct sockaddr*)&addr,
24         sizeof(addr));
25 write(s, "ciao", 5);
26 close(s);
```